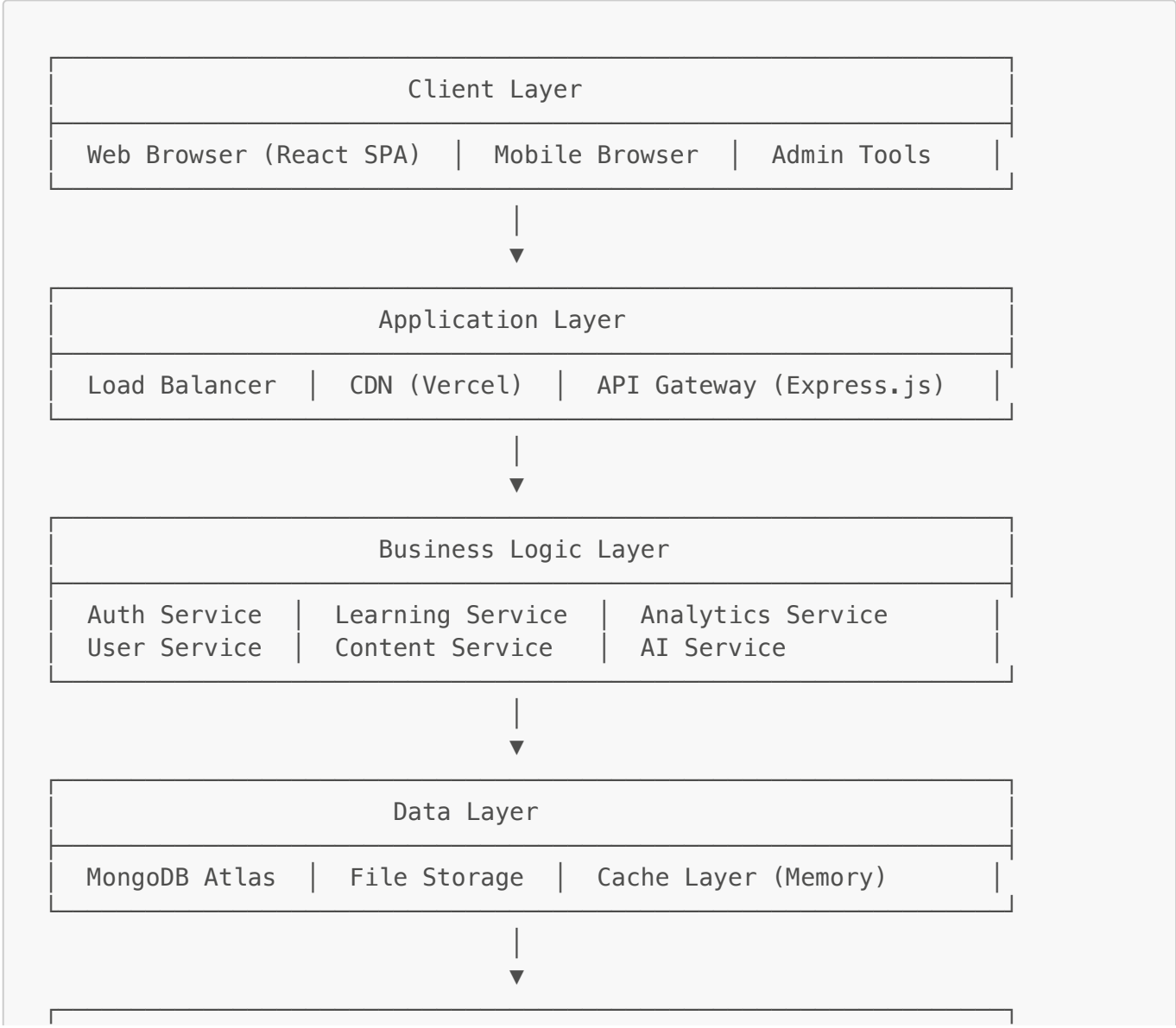
Syncner Technical Architecture

📋 Table of Contents

1. System Overview
2. Architecture Patterns
3. Frontend Architecture
4. Backend Architecture
5. Database Design
6. API Design
7. Security Architecture
8. Performance Architecture
9. Scalability Considerations
10. Technology Stack

🏗️ System Overview

High-Level Architecture



```
│                    External Services                        │
├─────────────────────────────────────────────────────────────┤
│  Google Gemini API  │  Email Service  │  Monitoring Services  │
└─────────────────────────────────────────────────────────────┘
```

Core Components

1. Frontend (React SPA)

- **Framework**: React 18 with TypeScript
- **State Management**: Redux Toolkit
- **Routing**: React Router v6
- **Styling**: Tailwind CSS with custom design system
- **Build Tool**: Vite
- **Deployment**: Vercel with CDN

2. Backend (Node.js API)

- **Runtime**: Node.js with Express.js
- **Authentication**: JWT-based
- **Validation**: Custom middleware
- **File Handling**: Multer for uploads
- **Deployment**: Render

3. Database (MongoDB)

- **Primary Database**: MongoDB Atlas
- **ODM**: Mongoose
- **Indexing**: Optimized for query performance
- **Backup**: Automated Atlas backups

4. External Services

- **AI Integration**: Google Gemini API
- **File Storage**: Local storage with cloud backup
- **Monitoring**: Built-in platform analytics

🎯 Architecture Patterns

1. Layered Architecture

```
┌─────────────────┐
│  Presentation   │ ← React Components, Pages
├─────────────────┤
│  Application    │ ← Redux Store, Services
├─────────────────┤
│  Business Logic │ ← Controllers, Services
├─────────────────┤
```

```
┌─────────────────────┐
│  Data Access        │ ← Models, Repositories
├─────────────────────┤
│  Database           │ ← MongoDB Collections
└─────────────────────┘
```

2. Microservices Pattern (Future)

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│ Auth Service │  │ Learning Svc │  │ Analytics Svc│
├──────────────┤  ├──────────────┤  ├──────────────┤
│ User Mgmt    │  │ Content Mgmt │  │ Reporting    │
│ JWT Auth     │  │ Progress     │  │ Metrics      │
│ Permissions  │  │ Assessment   │  │ Dashboards   │
└──────────────┘  └──────────────┘  └──────────────┘
```

3. Event-Driven Architecture

```
┌──────────────┐         Events        ┌──────────────┐
│   User       │ ──────────────────►   │  Analytics   │
│  Actions     │                       │  Service     │
└──────────────┘                       └──────────────┘
       │                                      │
       ▼                                      ▼
┌──────────────┐                       ┌──────────────┐
│  Progress    │                       │ Notifications│
│  Updates     │                       │  Service     │
└──────────────┘                       └──────────────┘
```

🎨 Frontend Architecture

Component Hierarchy

```
App
├── Layout
│   ├── Header
│   ├── Sidebar
│   └── Main Content
├── Pages
│   ├── Dashboard
│   ├── Learning
│   ├── Portfolio
│   └── Profile
├── Components
│   ├── UI Components
│   ├── Forms
│   └── Charts
```

```
└── Services
    ├── API Service
    ├── Auth Service
    └── Storage Service
```

State Management (Redux Toolkit)

```
// Store Structure
interface RootState {
  auth: AuthState;
  curriculum: CurriculumState;
  learningJourney: LearningJourneyState;
  portfolio: PortfolioState;
  staff: StaffState;
  theme: ThemeState;
  user: UserState;
}
```

Key Frontend Patterns

1. Container/Presentational Pattern

```
// Container Component
const StudentDashboardContainer = () => {
  const dispatch = useAppDispatch();
  const { user, progress } = useAppSelector(state => ({
    user: state.auth.user,
    progress: state.learningJourney.progress
  }));

  return <StudentDashboard user={user} progress={progress} />;
};

// Presentational Component
const StudentDashboard = ({ user, progress }) => {
  return (
    <div>
      {/* UI Implementation */}
    </div>
  );
};
```

2. Custom Hooks Pattern

```
// Custom Hook
const useStudentProgress = (careerTrack: string) => {
```

```
  const [progress, setProgress] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchProgress(careerTrack).then(setProgress).finally(() =>
setLoading(false));
  }, [careerTrack]);

  return { progress, loading };
};
```

3. Service Layer Pattern

```
// API Service
class ApiService {
  private baseURL: string;

  async get<T>(endpoint: string): Promise<T> {
    const response = await fetch(`${this.baseURL}${endpoint}`);
    return response.json();
  }

  async post<T>(endpoint: string, data: any): Promise<T> {
    const response = await fetch(`${this.baseURL}${endpoint}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });
    return response.json();
  }
}
```

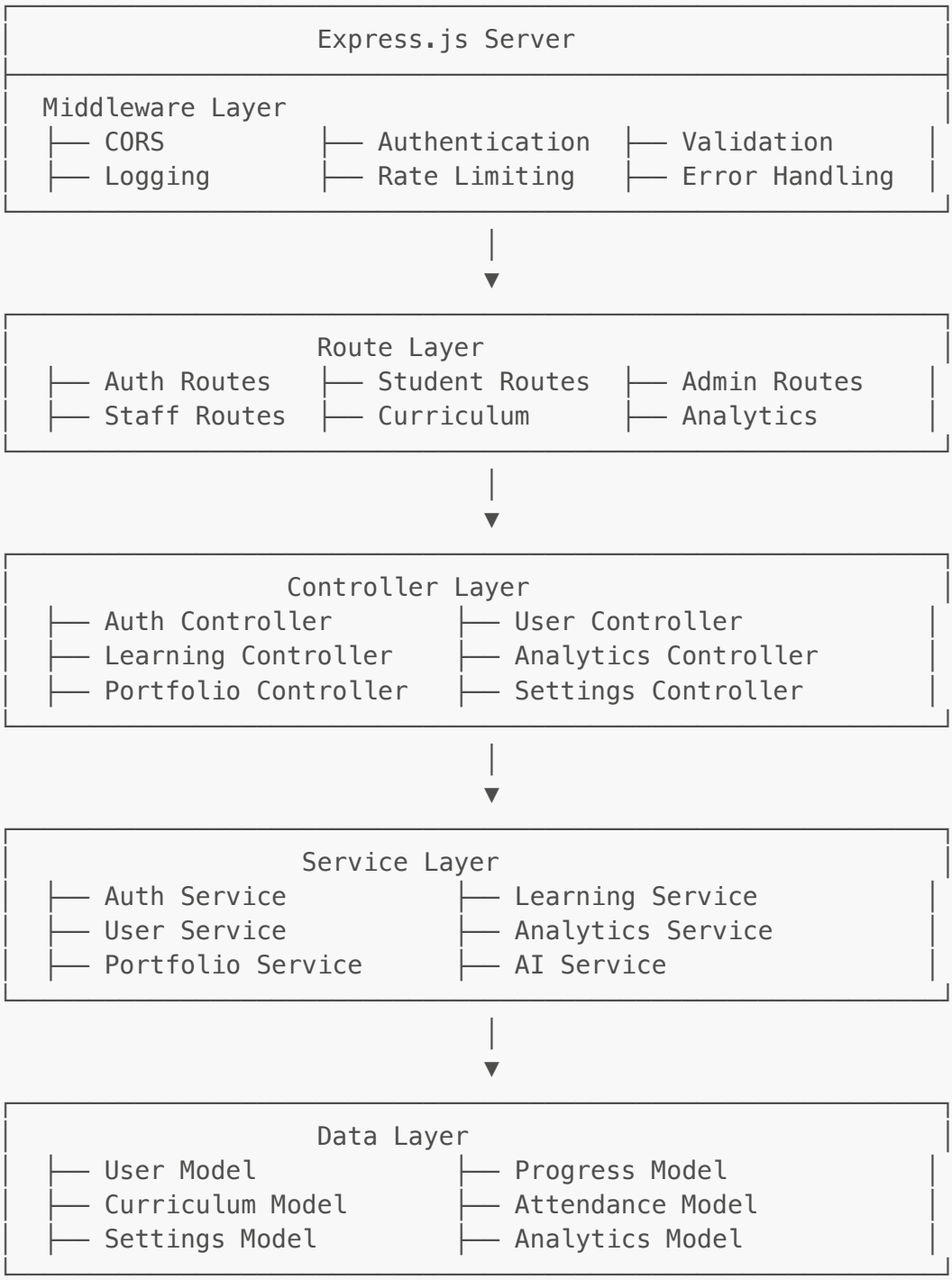
Design System Architecture

```
// Theme System
interface Theme {
  colors: {
    primary: string;
    secondary: string;
    accent: string;
    success: string;
    error: string;
    warning: string;
  };
  typography: {
    fontFamily: string;
    fontSize: Record<string, string>;
    fontWeight: Record<string, number>;
  };
```

```
  spacing: Record<string, string>;
  breakpoints: Record<string, string>;
}
```

⚙️ Backend Architecture

Service Layer Architecture



Key Backend Patterns

1. MVC Pattern

```
// Model
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  role: { type: String, enum: ['student', 'staff', 'admin'] }
});

// Controller
const createUser = async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json({ success: true, data: user });
  } catch (error) {
    res.status(400).json({ success: false, error: error.message });
  }
};

// Route
router.post('/users', protect, authorize('admin'), createUser);
```

2. Service Layer Pattern

```
// Service
class UserService {
  static async createUser(userData) {
    const user = await User.create(userData);
    await this.sendWelcomeEmail(user);
    return user;
  }

  static async updateUser(userId, updateData) {
    const user = await User.findByIdAndUpdate(userId, updateData, { new:
true });
    return user;
  }
}

// Controller
const createUser = async (req, res) => {
  const user = await UserService.createUser(req.body);
  res.status(201).json({ success: true, data: user });
};
```

3. Middleware Pattern

```
// Authentication Middleware
const protect = async (req, res, next) => {
```

```
  const token = req.header('Authorization')?.replace('Bearer ', '');

  if (!token) {
    return res.status(401).json({ error: 'Access denied' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    next();
  } catch (error) {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Authorization Middleware
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Access denied' });
    }
    next();
  };
};
```

Error Handling Architecture

```
// Global Error Handler
const errorHandler = (err, req, res, next) => {
  let error = { ...err };
  error.message = err.message;

  // Log error
  console.error(err);

  // Mongoose bad ObjectId
  if (err.name === 'CastError') {
    const message = 'Resource not found';
    error = new ErrorResponse(message, 404);
  }

  // Mongoose duplicate key
  if (err.code === 11000) {
    const message = 'Duplicate field value entered';
    error = new ErrorResponse(message, 400);
  }

  // Mongoose validation error
  if (err.name === 'ValidationError') {
    const message = Object.values(err.errors).map(val => val.message);
    error = new ErrorResponse(message, 400);
```

```
  }

  res.status(error.statusCode || 500).json({
    success: false,
    error: error.message || 'Server Error'
  });
};
```

🗄️ Database Design

Database Architecture

```
MongoDB Atlas Cluster
├── Primary Database (syncner)
│   ├── Collections
│   │   ├── users
│   │   ├── curriculum
│   │   ├── userprogress
│   │   ├── attendance
│   │   └── settings
│   └── Indexes
│       ├── Performance Indexes
│       ├── Unique Indexes
│       └── Compound Indexes
└── Backup Database
    └── Automated Backups
```

Collection Schemas

1. Users Collection

```
{
  _id: ObjectId,
  name: String,
  email: String (unique),
  password: String (hashed),
  role: String (enum: ['student', 'staff', 'admin']),
  careerTrack: String,
  college: String,
  profileComplete: Boolean,
  isActive: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

2. Curriculum Collection

```
{
  _id: ObjectId,
  title: String,
  description: String,
  careerTrack: String,
  order: Number,
  difficulty: String,
  duration: String,
  isActive: Boolean,
  learn: [{
    id: String,
    title: String,
    content: String,
    order: Number
  }],
  test: [{
    id: String,
    question: String,
    options: [String],
    answer: String,
    order: Number
  }],
  exercise: [{
    id: String,
    title: String,
    description: String,
    resources: [String],
    order: Number
  }]
}
```

3. UserProgress Collection

```
{
  _id: ObjectId,
  userId: ObjectId (ref: 'User'),
  careerTrack: String,
  completedLevels: [{
    levelId: ObjectId,
    levelOrder: Number,
    completedSections: {
      learn: Boolean,
      test: Boolean,
      exercise: Boolean
    },
    learnProgress: {
      timeSpent: Number,
      linksAccessed: [String],
      completed: Boolean
    },
```

```
    testProgress: {
      score: Number,
      attempts: Number,
      completed: Boolean
    },
    exerciseProgress: {
      submissions: [{
        links: [String],
        submittedAt: Date,
        status: String
      }],
      completed: Boolean
    }
  }]
}
```

Indexing Strategy

```
// Performance Indexes
db.users.createIndex({ email: 1 });
db.users.createIndex({ careerTrack: 1, role: 1 });
db.userprogress.createIndex({ userId: 1, careerTrack: 1 });
db.curriculum.createIndex({ careerTrack: 1, order: 1 });
db.attendance.createIndex({ userId: 1, date: 1 });

// Compound Indexes
db.userprogress.createIndex({
  userId: 1,
  "completedLevels.levelOrder": 1
});
```

🔌 API Design

RESTful API Architecture

```
Base URL: https://api.syncner.com/v1

┌─────────────────────────────────────────────────────────────┐
│                    API Endpoints                            │
├─────────────────────────────────────────────────────────────┤
│  Authentication                                             │
│  POST   /auth/login          POST   /auth/register          │
│  GET    /auth/me             POST   /auth/logout            │
├─────────────────────────────────────────────────────────────┤
│  User Management                                            │
│  GET    /users               POST   /users                  │
│  GET    /users/:id           PUT    /users/:id              │
│  DELETE /users/:id           POST   /users/:id/reset-pwd    │
├─────────────────────────────────────────────────────────────┤
```

```
│  Learning Management                                          │
│  GET    /curriculum         POST   /curriculum/levels        │
│  GET    /progress/user/:track POST  /progress/complete       │
│  POST   /progress/test      POST   /progress/exercise        │
├──────────────────────────────────────────────────────────────┤
│  Staff Management                                            │
│  GET    /staff/students     GET    /staff/students/:id       │
│  GET    /staff/analytics    GET    /staff/portfolio/:id      │
└──────────────────────────────────────────────────────────────┘
```

API Design Principles

1. RESTful Design

- **Resource-based URLs**: `/users`, `/curriculum`, `/progress`
- **HTTP Methods**: GET, POST, PUT, DELETE for CRUD operations
- **Status Codes**: Proper HTTP status codes for responses
- **JSON Format**: Consistent JSON request/response format

2. API Versioning

```
// URL Versioning
app.use('/api/v1', v1Routes);
app.use('/api/v2', v2Routes);

// Header Versioning
app.use((req, res, next) => {
  const version = req.headers['api-version'] || 'v1';
  req.apiVersion = version;
  next();
});
```

3. Response Format

```
// Success Response
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully"
}

// Error Response
{
  "success": false,
  "error": "Error message",
  "code": "ERROR_CODE",
  "details": { ... }
}
```

API Security

1. Authentication Flow

```
// JWT Token Structure
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "id": "user_id",
    "role": "student",
    "iat": 1640995200,
    "exp": 1643587200
  }
}
```

2. Rate Limiting

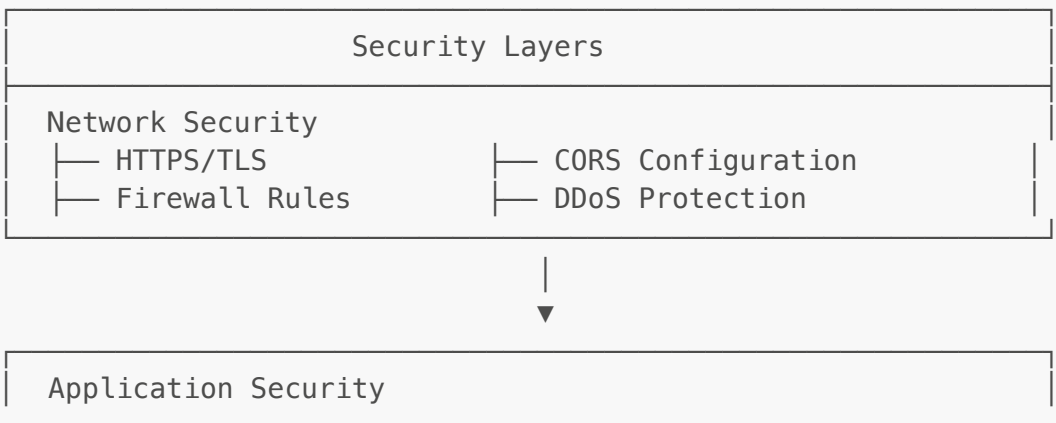
```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP'
});

app.use('/api/', limiter);
```

🔒 Security Architecture

Security Layers



```
│   ├── JWT Authentication     ├── Role-based Authorization   │
│   ├── Input Validation       ├── SQL Injection Prevention   │
│   ├── XSS Protection         ├── CSRF Protection            │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  Data Security                                               │
│   ├── Password Hashing       ├── Data Encryption            │
│   ├── Secure Storage         ├── Backup Encryption          │
└──────────────────────────────────────────────────────────────┘
```

Authentication & Authorization

```
// Password Hashing
const bcrypt = require('bcryptjs');
const saltRounds = 12;

const hashPassword = async (password) => {
  return await bcrypt.hash(password, saltRounds);
};

// JWT Token Generation
const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRE
  });
};

// Role-based Access Control
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Access denied' });
    }
    next();
  };
};
```

Data Protection

```
// Input Sanitization
const validator = require('validator');
const xss = require('xss');

const sanitizeInput = (input) => {
  return xss(validator.escape(input));
};
```

```
// File Upload Security
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() *
1E9);
    cb(null, file.fieldname + '-' + uniqueSuffix +
path.extname(file.originalname));
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = ['image/jpeg', 'image/png', 'application/pdf'];
  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type'), false);
  }
};
```

⚡ Performance Architecture

Performance Optimization Strategies

1. Frontend Performance

```
// Code Splitting
const LazyComponent = React.lazy(() => import('./LazyComponent'));

// Memoization
const MemoizedComponent = React.memo(({ data }) => {
  return <div>{data}</div>;
});

// Virtual Scrolling
import { FixedSizeList as List } from 'react-window';

const VirtualizedList = ({ items }) => (
  <List
    height={600}
    itemCount={items.length}
    itemSize={50}
    itemData={items}
  >
    {({ index, style, data }) => (
      <div style={style}>
```

```
          {data[index]}
        </div>
      )}
    </List>
  );
```

2. Backend Performance

```
// Database Query Optimization
const getStudentsWithProgress = async (filters) => {
  return await User.aggregate([
    { $match: filters },
    {
      $lookup: {
        from: 'userprogress',
        localField: '_id',
        foreignField: 'userId',
        as: 'progress'
      }
    },
    {
      $project: {
        name: 1,
        email: 1,
        careerTrack: 1,
        progress: { $arrayElemAt: ['$progress', 0] }
      }
    }
  ]);
};

// Caching Strategy
const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 600 }); // 10 minutes

const getCachedData = async (key, fetchFunction) => {
  let data = cache.get(key);
  if (!data) {
    data = await fetchFunction();
    cache.set(key, data);
  }
  return data;
};
```

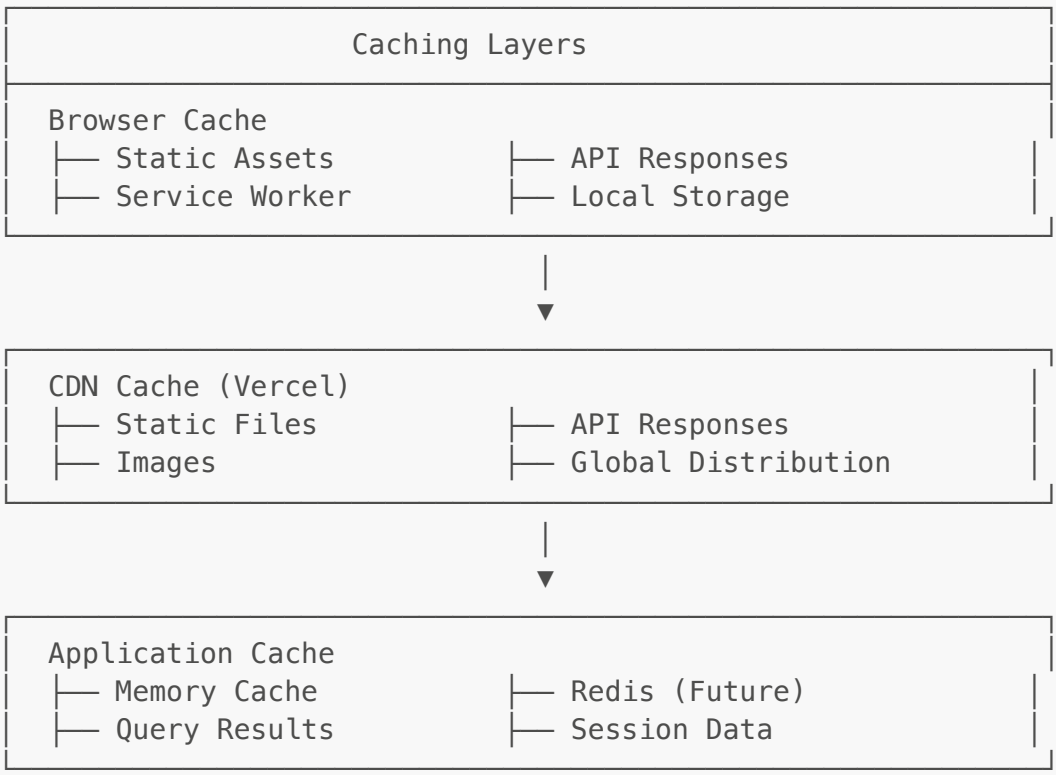
3. Database Performance

```
// Connection Pooling
mongoose.connect(process.env.MONGODB_URI, {
```

```
  maxPoolSize: 10,
  serverSelectionTimeoutMS: 5000,
  socketTimeoutMS: 45000,
});

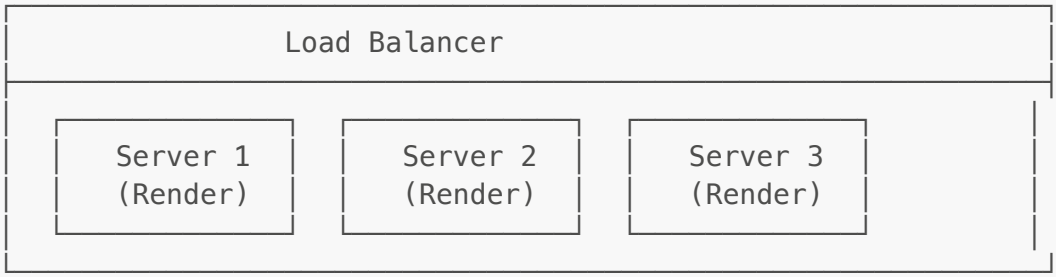
// Query Optimization
const optimizedQuery = async (userId) => {
  return await UserProgress.findOne({ userId })
    .select('completedLevels.levelOrder
completedLevels.completedSections')
    .lean(); // Returns plain JavaScript object
};
```

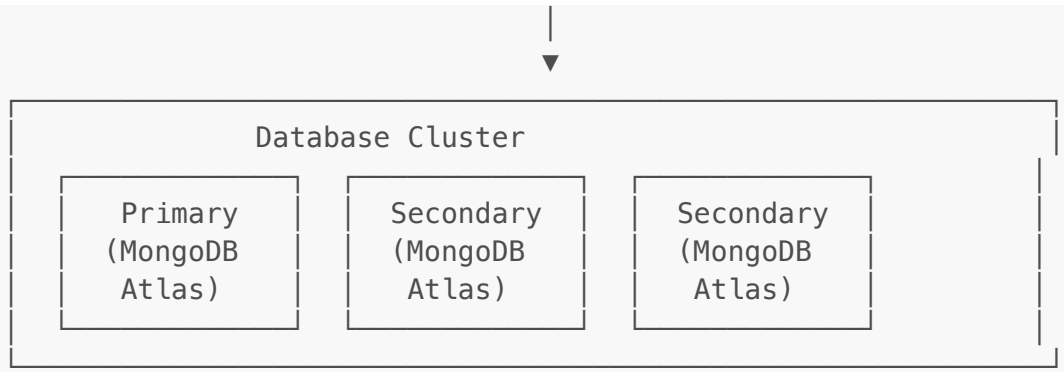
Caching Architecture



📈 Scalability Considerations

Horizontal Scaling Strategy





Microservices Migration Path

```
Current Monolith → Service Decomposition → Microservices

Phase 1: Extract Services
├── Authentication Service
├── User Management Service
└── Learning Service

Phase 2: Add Services
├── Analytics Service
├── Notification Service
└── File Storage Service

Phase 3: Advanced Services
├── AI Service
├── Reporting Service
└── Integration Service
```

Database Scaling

```
// Read Replicas
const readDB = mongoose.createConnection(process.env.MONGODB_READ_URI);
const writeDB = mongoose.createConnection(process.env.MONGODB_WRITE_URI);

// Sharding Strategy (Future)
const shardKey = { userId: 1, careerTrack: 1 };
db.userprogress.createIndex(shardKey);
```

🛠️ Technology Stack

Frontend Stack

```
{
  "framework": "React 18.2.0",
```

```
  "language": "TypeScript 4.9.5",
  "stateManagement": "Redux Toolkit 1.9.7",
  "routing": "React Router 6.8.1",
  "styling": "Tailwind CSS 3.2.7",
  "animations": "Framer Motion 10.12.4",
  "buildTool": "Vite 4.1.0",
  "testing": "Vitest 0.29.2",
  "linting": "ESLint 8.35.0",
  "formatting": "Prettier 2.8.4"
}
```

Backend Stack

```
{
  "runtime": "Node.js 18.15.0",
  "framework": "Express.js 4.18.2",
  "language": "JavaScript ES2022",
  "database": "MongoDB 6.0.4",
  "odm": "Mongoose 7.0.1",
  "authentication": "JWT 9.0.0",
  "validation": "Joi 17.8.3",
  "fileUpload": "Multer 1.4.5",
  "security": "bcryptjs 2.4.3",
  "cors": "cors 2.8.5",
  "helmet": "helmet 6.0.1"
}
```

DevOps Stack

```
{
  "frontendHosting": "Vercel",
  "backendHosting": "Render",
  "database": "MongoDB Atlas",
  "cdn": "Vercel Edge Network",
  "monitoring": "Built-in Analytics",
  "ci/cd": "GitHub Actions",
  "versionControl": "Git",
  "packageManager": "npm"
}
```

External Services

```
{
  "ai": "Google Gemini API",
  "email": "SMTP (Future: SendGrid)",
  "storage": "Local + Cloud Backup",
```

```
  "monitoring": "Platform Analytics",
  "logging": "Console + File Logs"
}
```

🔄 Development Workflow

Git Workflow

```
main (production)
├── develop (staging)
│   ├── feature/user-authentication
│   ├── feature/learning-progress
│   └── feature/portfolio-system
└── hotfix/critical-bug-fix
```

CI/CD Pipeline

```
# .github/workflows/ci.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      - name: Install dependencies
        run: |
          cd src && npm install
          cd ../backend && npm install
      - name: Run tests
        run: |
          cd src && npm test
          cd ../backend && npm test
      - name: Build applications
        run: |
          cd src && npm run build
          cd ../backend && npm run build
```

Code Quality Standards

```
{
  "eslint": {
    "extends": ["@typescript-eslint/recommended"],
    "rules": {
      "no-console": "warn",
      "no-unused-vars": "error",
      "prefer-const": "error"
    }
  },
  "prettier": {
    "semi": true,
    "trailingComma": "es5",
    "singleQuote": true,
    "printWidth": 80
  }
}
```

Architecture Version: v1.0
Submitted On: September 2025